

Projet ASD : Structures de Données Hybrides

Seydina Mohamed Diop & Abdou Lahate

13 juin 2026

1 Introduction

Ce Projet porte sur un systeme de gestion de scolarité . En efet on va consevoire une programme qui va permetre a un utilisateur de rensegne quelque information qui saffichera sur le terminal

2 Modélisation des Structures de Données

Voici les Structures de base que nous avons conçue pour notre Systeme De Scolarité.

```
1 typedef struct Note
2 {
3     char matier[20]; // ici on donne la matiere
4     float valeur; // sa valeur
5     int coef;
6     struct Note *note_suivante; // permet de passer au note
        suivant du meme etudiant
7 }Note;
```

2.0.1 Explication de la structure Note

La structure **Note** reçoit et stocke les resultats academiques d'un etudiant pour chaque matière de la filiere, ainsi que le coefficient correspondant. Par exemple, pour un étudiant donné, cette structure va enregistrer ses notes au fur et à mesure. Ensuite, grace au pointeur **struct Note *note_suivante**, la structure se connecte à la note suivante du même étudiant. C'est ce mécanisme qui forme une liste chaînée de notes propre à chaque etudiant, permettant de stocker un nombre dynamique de modules sans utiliser de tableau statique.

```
1 typedef struct Etudiant
2 {
```

```

3     char nom[50];
4     char prenom[50];
5     int numero_dossier; // identifiant unique par etudiant
6     bool eta_inscription; // 1 si l etudiant s'est inscrit et
        0 sinon (initialise lors de la creation)
7     float moyenne_general;
8     Note *modules; // est un pointeur de type Note
9     struct Etudiant *etudiant_suivant; // permet de passer a
        l etudiant suivant

```

2.0.2 Explication de la structure Etudiant

La structure `Etudiant` regroupe l'ensemble des informations d'un étudiant de la filière, telles que son nom, son prénom, son numéro de dossier (identifiant unique) et son état d'inscription (via un booleen). Le pointeur `modules` est essentielle : il pointe vers la première note de l'étudiant, liant ainsi l'étudiant à ses résultats. Concernant le pointeur `etudiant_suivant` elle pointe sur l'étudiant suivante créent ainsi une liste chaînée

```

1 }Etudiant;
2 typedef struct
3 {
4     char nom_filiere[60];
5     int nb_etudiants;
6     Etudiant *premier_du_liste; // point sur le premier de la
        liste des etudiant
7     Etudiant *dernier_du_liste; // point sur le dernier
        etudiant de la liste

```

2.0.3 Explication de la structure Filiere

Cette structure contient le nom d'une filière donner ainsi que le nombre d'étudiants. Dans la structure `Filiere` on a deux pointeurs : `premier_du_liste` qui pointe sur le premier étudiant de la liste du filière, `dernier_du_liste` quant à lui pointe sur le dernier étudiant. Permettant d'insérer des étudiants directement à la fin de la liste

3 Modélisation des Fonctions

Voici les fonctions que nous avons conçues pour permettre le bon fonctionnement de notre Système de scolarité.

```

1 Etudiant *inscrire_etudiants(Filiere *filiere)
2 {
3     // allocation dynamique d un nouveau etudiant

```

```

4      Etudiant *nouveau_etudiant = (Etudiant*) malloc(sizeof(
      Etudiant));
5      if (nouveau_etudiant == NULL)
6      {
7          printf("Erreur : impossible d'allouer la memoire");
8          return;
9      }
10     // ajout des information d un etudiant par l utilisateur
11     printf("\n-----Formulaire d inscription-----\n");
12     printf("Donner le nom de l etudiant : \n");
13     scanf("%s", nouveau_etudiant->nom);
14     printf("Donner le prenom de l etudiant : \n");
15     scanf("%s", nouveau_etudiant->prenom);
16     printf("Donner son numero de dossier : \n");
17     scanf("%d", &nouveau_etudiant->numero_dossier);
18
19     nouveau_etudiant->eta_inscription = 1;    // on va
        enregistrer l etat d inscription a 1 par default
20     nouveau_etudiant->modules = NULL ; // pas encore de note
21     nouveau_etudiant->etudiant_suivant = NULL; // se sera le
        dernier etudiant
22
23     if (filiere->premier_du_liste == NULL)
24     {
25         // La liste est vide, il est le tout premier et le
            tout dernier
26         filiere->premier_du_liste = nouveau_etudiant;
27         filiere->dernier_du_liste = nouveau_etudiant;
28     } else
29     {
30         filiere->dernier_du_liste->etudiant_suivant =
            nouveau_etudiant; // le pointeur dernier_du_liste
            pointe sure le nouveau etudiant
31         filiere->dernier_du_liste = nouveau_etudiant; // Il
            devient le nouveau dernier de la filiere
32     }
33
34     filiere->nb_etudiants++; // On incremente le compteur de
        la classe
35     printf("L'etudiant %s %s a ete inscrit avec succes!\n",
        nouveau_etudiant->prenom, nouveau_etudiant->nom);
36     return nouveau_etudiant;
37 }

```

3.0.1 Explication de la Fonctions inscrire_etudiant

Cette fonction nous permet d ajouter de nouveaux etudiant dans une filiere en enregistrant leur information personnel : prenom, nom, numero de dos-

sier grace a l allocation dynamique suivant :`Etudiant *nouveau_etudiant = (Etudiant*) malloc(sizeof(Etudiant))` qui allout une espace memoire dans le memoire RAM. Combiner au boucle suivant.

```

1   if (filiere->premier_du_liste == NULL)
2   {
3       // La liste est vide, il est le tout premier et le
          tout dernier
4       filiere->premier_du_liste = nouveau_etudiant;
5       filiere->dernier_du_liste = nouveau_etudiant;
6   } else
7   {
8       filiere->dernier_du_liste->etudiant_suivant =
          nouveau_etudiant; // le pointeur dernier_du_liste
          pointe sure le nouveau etudiant
9       filiere->dernier_du_liste = nouveau_etudiant; // Il
          devient le nouveau dernier de la filiere
10  }
```

l utilisateur pourra ajouter autant d etudiant qu il souhaite, le programme aura ainsi le nombre totale d etudiants.

```

1 void note_etudiants(Etudiant *etudiant) // une fonction
    permettant d enregistrer les resultat academique d un
    etudiant
2 {
3     if (etudiant == NULL) return;
4
5     // On verifie si l'etudiant est bien inscrit
6     if (etudiant->eta_inscription == 0)
7     {
8         printf("Impossible d'ajouter une note : cet etudiant
          n'est pas activement inscrit.\n");
9         return;
10    }
11    int nouveaux_matiere = 2;
12    while (nouveaux_matiere == 2)
13    {
14        Note *nouveau_note = (Note*)malloc(sizeof(Note)); //
          allocation dynamique
15        if (nouveau_note == NULL)
16        {
17            printf("Ereure : impossible d allouer la memoire");
18            return;
19        }
20
21
22        // ajout du resultat d un etudiant
23        printf("\n-----Resultats Academique-----\n");
24        printf("Donner la matiere : \n");
```

```

25     scanf("%s", nouveau_note->matier);
26     do
27     {
28         printf("Donner la note obtenue: \n");
29         scanf("%f", &nouveau_note->valeur);
30         if (nouveau_note->valeur < 0 || nouveau_note->
            valeur > 20)
31         {
32             printf("Erreur! La note doit etre comprise
                entre 0 et 20. \n");
33         }
34     } while (nouveau_note->valeur < 0 || nouveau_note->
        valeur > 20);
35     do
36     {
37         printf("Donner le coefficient du matiere: \n");
38         scanf("%d", &nouveau_note->coef);
39         if (nouveau_note->coef <= 0 || nouveau_note->coef
            > 10)
40         {
41             printf("Erreur! Le coefficient doit etre
                compris entre 1 et 10. \n");
42         }
43     } while (nouveau_note->coef <= 0 || nouveau_note->
        coef > 10);
44
45     nouveau_note->note_suivante = NULL; // c est la
        premiere et dernier note
46
47     if (etudiant->modules == NULL)
48     {
49         // Cas ou l etudiant n a encore aucune note
50         etudiant->modules = nouveau_note;
51     }
52     else
53     {
54         // Cas ou l'etudiant a deja des notes, on
            parcourt pour aller a la fin
55         Note *note_restante = etudiant->modules;
56         while (note_restante->note_suivante != NULL)
57         {
58             note_restante = note_restante->note_suivante;
59         }
60         // On accroche la nouvelle note tout a la fin
61         note_restante->note_suivante = nouveau_note;
62     }
63     printf("Matiere ajouter avec succe: \n");
64     printf("Si vous voulez ajouter une nouvelle matiere
        tape 2 sinon 0: \n");

```

```

65         scanf("%d", &nouveaux_matiere);
66     }
67 }

```

3.0.2 Explication de la Fonctions note_etudiant

Comme la fonction precedente celui ci Permet de stocke les resultats scolaire de chaque etudiants enregistrer en amont ,grace a l allocation dynamique suivant `Note *nouveau_note = (Note*)malloc(sizeof(Note))`. Cette fonction va permettre a l utilisateur de donner autant de matiere ainsi que les note et coefficient associer. Pour chacune des etudiants qu il enregistre il peut aussi ne pas ajouter de note pour les etudiants qu il souhaite . Et cela grace au boucle `while (nouveaux_matiere == 2)` qui lui dira de taper 2 s il veut ajouter des matiere et 0 sinon .

```

1 void liste_de_la_classe(Filiere *filiere)
2 {
3     if (filiere == NULL || filiere->premier_du_liste == NULL)
4         // verifie si la filiere est vide ou non
5     {
6         printf("Aucun etudiant inscrit dans la filiere.\n");
7         return;
8     }
9     printf("\n-----Liste des etudiants de la classe\n");
10    Etudiant *actuel_liste = filiere->premier_du_liste;
11    while (actuel_liste != NULL) // affiche les identifiant
12        des etudiants
13    {
14        printf("Nom: %s, Prenom: %s, Numero de dossier: %d\n",
15              ,actuel_liste->nom, actuel_liste->prenom,
16              actuel_liste->numero_dossier);
17        actuel_liste = actuel_liste->etudiant_suivant;
18    }
19 }

```

3.0.3 Explication de la Fonctions liste_de_la_classe

Quant a cette fonction il affichera la liste des etudiants que l utilisateur a deja enregistrer .Il donne respectivement leur nom, prenom et numero de dossier.

```

1 void bulletin(Filiere *filiere)
2 {
3     if (filiere == NULL || filiere->premier_du_liste == NULL)

```

```

4      {
5          printf("Aucun étudiant inscrit dans la filiere.\n");
6          return;
7      }
8
9      printf("\n-----Bulletin des étudiants de la classe
          -----\n");
10
11     // VARIABLES AJOUTEES POUR LA MOYENNE GENERALE (Ligne 1
        et 2)
12     float somme_des_moyennes_classe = 0;
13     int etudiants_avec_notes = 0; // prend en compte que les
        étudiants ayant une moyenne
14
15     Etudiant *actuel_liste_info = filiere->premier_du_liste;
16
17     while (actuel_liste_info != NULL)
18     {
19         // On verifie le statut de chaque etudiant
        individuellement ici
20         if (actuel_liste_info->eta_inscription == 1)
21         {
22             printf("Nom: %s, Prenom: %s, Numero de dossier: %d\n",
                    actuel_liste_info->nom,
                    actuel_liste_info->prenom, actuel_liste_info->
                    numero_dossier);
23             Note *actuel_liste_bulletin = actuel_liste_info->
                    modules;
24             float point_totale = 0;
25             int coef_totale = 0;
26
27             while (actuel_liste_bulletin != NULL)
28             {
29                 printf("Matiere: %s, Note: %f, Coeficiant: %d\n",
                        actuel_liste_bulletin->matier,
                        actuel_liste_bulletin->valeur,
                        actuel_liste_bulletin->coef);
30                 point_totale += actuel_liste_bulletin->valeur
                        * actuel_liste_bulletin->coef;
31                 coef_totale += actuel_liste_bulletin->coef;
32                 actuel_liste_bulletin = actuel_liste_bulletin
                        ->note_suivante;
33             }
34
35             if (coef_totale != 0)
36             {
37                 actuel_liste_info->moyenne_general =
                        point_totale / coef_totale;

```

```

38         printf("Moyenne_Generale: %.2f\n\n",
39                actuel_liste_info->moyenne_general);
40         somme_des_moyennes_classe +=
41         actuel_liste_info->moyenne_general;
42         etudiants_avec_notes++;
43     }
44     else
45     {
46         actuel_liste_info->moyenne_general = 0;
47         printf("Aucune_note_enregistre\n");
48     }
49     // On passe l' etudiant suivant
50     actuel_liste_info = actuel_liste_info->
51     etudiant_suivant;
52 }
53 // Affichage de la moyenne gennerale
54 if (etudiants_avec_notes != 0)
55 {
56     float moyenne_general_classe =
57     somme_des_moyennes_classe / etudiants_avec_notes;
58     printf("MOYENNE_GENERALE_DE_LA_FILIERE: %.2f\n",
59            , moyenne_general_classe);
60 }
61 else
62 {
63     printf("Les etudiants non pas de note.\n");
64 }

```

3.0.4 Explication de la Fonctions bulletin

La fonction bulletin permet de calculer les moyenne de chaque etudiant ainsi que la moyenne gennerale de la classe. Elle va aller chercher les module de chaque etudiants c est a dire ces matiere ainsi que les coef et notes associer pour calculer sa moyenne. Si l etudiant n a pas de note la boucle :

```

1         else
2         {
3             actuel_liste_info->moyenne_general = 0;
4             printf("Aucune_note_enregistre\n");
5         }

```

Va afficher a l ecran que cette etudiant n a pas de note ,cette etudiant ne sera pas prise en compte pour le calcule du moyenne gennerale de la classe . Grace au variable `etudiants_avec_notes` , innicialise a 0 qui n avance que

si l etudiant qu on doit calculer la moyenne a une note .De cette facon avec la boucle :

```

1   if (etudiants_avec_notes != 0)
2   {
3       float moyenne_general_classe =
          somme_des_moyennes_classe / etudiants_avec_notes;
4       printf("MOYENNE GENERALE DE LA FILIERE: %.2f / 20\n"
          , moyenne_general_classe);
5   }
6   else
7   {
8       printf("Les etudiants non pas de note.\n");
9   }
10 }

```

Cqlcule et affiche la moyenne gennerale exacte de la classe.

```

1 void recherche_et_modification_etudiant(Filiere *filiere)
2 {
3     int votre_choix = 3;
4     while (votre_choix == 3)
5     {
6         printf("Voulez vous rechercher ou modifier les info d
          un autre etudiant si oui taper 3 si non 0:\n");
7         scanf("%d", &votre_choix);
8         if (votre_choix == 0)
9         {
10            break;
11        }
12        if (filiere == NULL || filiere->premier_du_liste ==
            NULL)
13        {
14            printf("Aucun etudiant dans la filiere.\n");
15            return;
16        }
17        int id_rechercher;
18        printf("Donner le numero de dossier de l etudiant
          rechercher:\n");
19        scanf("%d", &id_rechercher);
20
21        Etudiant *etudiant_actuel = filiere->premier_du_liste
          ;
22        while (etudiant_actuel != NULL && etudiant_actuel->
          numero_dossier != id_rechercher)
23        {
24            etudiant_actuel = etudiant_actuel->
          etudiant_suivant;
25        }
26        if (etudiant_actuel == NULL)

```

```

27     {
28         printf("%d ne correspond a aucun etudiant
           enregistre:\n", id_rechercher);
29     }
30     else
31     {
32         int choix;
33         printf("\n-----Etudiant trouver-----\n");
34         printf("Nom: %s, Prenom: %s, Numero de dossier: %d,
           Etat d'inscription: %s", etudiant_actuel->
           nom, etudiant_actuel->prenom, etudiant_actuel
           ->numero_dossier, (etudiant_actuel->
           eta_inscription == 1) ? "Inscrit" : "Non
           inscrit");
35         printf("\n-----Que souhaitez-vous modifier
           -----\n");
36         printf("1 Le Nom\n");
37         printf("2 Le Prenom\n");
38         printf("3 Le Numero de dossier\n");
39         printf("4 L'etat d'inscription (1: inscrit, 0:
           Suspendu)\n");
40         printf("5 Matiere Note et Coef\n");
41         printf("Votre choix: ");
42         scanf("%d", &choix);
43
44         switch (choix) // cette partie va permettre a l
           utilisateur de modifier les parametre qu il
           souhaite
45         {
46             case 1:
47                 printf("Entrez le nouveau nom: \n");
48                 scanf("%s", etudiant_actuel->nom);
49                 printf("Nom modifie avec succes!\n");
50                 break;
51             case 2:
52                 printf("Entrez le nouveau prenom: \n");
53                 scanf("%s", etudiant_actuel->prenom);
54                 printf("Prenom modifie avec succes!\n");
55                 break;
56             case 3:
57                 printf("Entrez le nouveau numero de
           dossier: \n");
58                 scanf("%d", &etudiant_actuel->
           numero_dossier);
59                 printf("Numero de dossier modifie avec
           succes!\n");
60                 break;
61             case 4:
62                 printf("Entrez le nouveau statut (1 pour

```

```

63         Inscrit, 0 pour Non inscrit) : \n");
scanf("%d", &etudiant_actuel->
    eta_inscription);
64     printf("Statut d'inscription mis a jour
        !\n");
65     case 5:
66         printf("Entrez la nouvelle matiere : \n");
67         scanf("%s", &etudiant_actuel->modules->
            matier);
68         printf("Entrez la nouvelle note : \n");
69         scanf("%f", &etudiant_actuel->modules->
            valeur);
70         printf("Entrez le nouveau coef : \n");
71         scanf("%d", &etudiant_actuel->modules->
            coef);
72         printf("La matiere ainsi que sa note et
            son coef on ete modifier avec succe !\n");
73         break;
74         default:
75             printf("Choix invalide. Aucune
                modification effectuee. \n");
76             break;
77     }
78
79 }
80 }
81
82 }

```

3.0.5 Explication de la Fonctions recherche_et_modification_etudiant

Cette fonction va demander a l'utilisateur s'il veut rechercher un etudiant afin de modifier ou non ces informations personnelles ou académiques en lui affichant un message lui permettant de faire deux choix soit taper 3 pour continuer et avoir la possibilité de modifier certaines informations. Soit taper 0 pour arrêter le programme. et éventuellement ne rien modifier ou sortir après des modifications.

```

1 void supprimer_etudiant(Filiere *filiere)
2 {
3     int votre_choix = 4;
4     while (votre_choix == 4)
5     {
6         printf("Si vous voulez supprimer une autre etudiant
            tape 4 sinon 0 : \n");
7         scanf("%d", &votre_choix);
8         if (votre_choix == 0)

```

```

9      {
10          break;
11      }
12
13      if (filiere == NULL || filiere->premier_du_liste ==
14          NULL)
15      {
16          printf("Il n'y a aucune etudiant");
17          return;
18      }
19      int id_supprimer;
20      printf("Donner le numero de dossier de l'etudiant que
21          vous souhaitez supprimer de la filiere:\n");
22      scanf("%d", &id_supprimer);
23      Etudiant *actuel_etudiant = filiere->premier_du_liste
24          ;
25      Etudiant *precedent_etudiant = NULL;
26      while (actuel_etudiant != NULL && id_supprimer !=
27          actuel_etudiant->numero_dossier)
28      {
29          precedent_etudiant = actuel_etudiant; // l
30          etudiant precedent devient l actuel etudiant
31          actuel_etudiant = actuel_etudiant->
32          etudiant_suivant; // l actuel etudiant pas a l
33          etudiant suivant
34      }
35      if (actuel_etudiant == NULL)
36      {
37          printf("Etudiant introuvable avec l'id: %d\n",
38              id_supprimer);
39      }
40      else
41      {
42          if (precedent_etudiant == NULL)
43          {
44              // correspond au premier etudiant
45              filiere->premier_du_liste = actuel_etudiant->
46              etudiant_suivant;
47              // cas ou la liste contient q un seul etudiant
48              if (filiere->premier_du_liste == NULL)
49              {
50                  filiere->dernier_du_liste = NULL;
51              }
52          }
53          else
54          {
55              // l etudiant est au milieux ou a la fin

```

```

48         precedent_etudiant->etudiant_suivant =
           actuel_etudiant->etudiant_suivant;
49         // Si on a supprimé le dernier, l'avant-
           dernier (precedent) devient le nouveau
           dernier
50         if (actuel_etudiant == filiere->
           dernier_du_liste)
51         {
52             filiere->dernier_du_liste =
           precedent_etudiant;
53         }
54     }
55     free(actuel_etudiant); // libérer l'espace alloué
           à l'étudiant supprimé
56     filiere->nb_etudiants--;
57     printf("L'étudiant avec l'id : %d a été supprimé
           avec succès\n", id_supprimer);
58 }
59 }
60 }

```

3.0.6 Explication de la Fonction supprimer_etudiant

De la même manière que la fonction précédente celui-ci lui permettra de supprimer ou non un étudiant de la liste

```

1 void maximum(Filiere *filiere)
2 {
3     if (filiere == NULL || filiere->premier_du_liste == NULL)
4     {
5         printf("Il n'y a aucune étudiant");
6         return;
7     }
8     float majorant = 0.0;
9     Etudiant *etudiant_major = filiere->premier_du_liste;
10    Etudiant *le_major = NULL; // Pointeur pour mémoriser le
           majorant
11    while (etudiant_major != NULL)
12    {
13        if (etudiant_major->eta_inscription == 1 &&
           etudiant_major->moyenne_general >= 0.0)
14        {
15            if (etudiant_major->moyenne_general > majorant)
16            {
17                majorant = etudiant_major->moyenne_general;
           // mise à jour du nouveau majorant
18                le_major = etudiant_major; // mémoriser l'
           emplacement du majorant actuelle

```

```

19         }
20     }
21     etudiant_major = etudiant_major->etudiant_suivant;
22 }
23 if (le_major != NULL)
24 {
25     printf("Le_majorant_de_la_filiere:%s,est%s,%s,
           numero_de_dossier:%d,et_a_une_moyenne_generale
           de:%.2f\n",filiere->nom_filiere, le_major->prenom
           , le_major->nom, le_major->numero_dossier,
           le_major->moyenne_general);
26 }
27 }

```

3.0.7 Explication de la Fonctions maximum

Cette fonction va chercher parmit tout les etudiant celui qui a la plus grande moyenne et a la fin du programme il donne sont nom prenom numero de dossier sa moyenne et le nom de la filiere.

```

1 void minimum(Filiere *filiere)
2 {
3     if (filiere == NULL || filiere->premier_du_liste == NULL)
4     {
5         printf("Il_n_y_a_aucune_etudiant");
6         return;
7     }
8     float minorant = 20;
9     Etudiant *etudiant_minor = filiere->premier_du_liste;
10    Etudiant *le_minor = NULL; // Pointeur pour m moriser le
        minorant
11    while (etudiant_minor != NULL)
12    {
13        if (etudiant_minor->eta_inscription == 1 &&
            etudiant_minor->moyenne_general >= 0.0)
14        {
15            if (etudiant_minor->moyenne_general < minorant)
16            {
17                minorant = etudiant_minor->moyenne_general;
18                // mise a jour du nouveaux minorant
19                le_minor = etudiant_minor; // memorise l
                enplacement du minorant actuelle
20            }
21            etudiant_minor = etudiant_minor->etudiant_suivant;
22        }
23        if (le_minor != NULL)
24        {

```

```

25     printf("L'etudiant ayant la plus faible note de la  

        filiere:%s,est%s,%s,numero de dossier:%d,  

        et a une moyenne generale de: %.2f\n",filiere->  

        nom_filiere, le_minor->prenom, le_minor->nom,  

        le_minor->numero_dossier, le_minor->  

        moyenne_general);
26 }
27 }

```

3.0.8 Explication de la Fonctions minimum

Pareil pour la fonction minimum ,il donne l etudiant ayant la plus faible moyenne de la filiere .

```

1 void du_majorant_minorant(Filiere *filiere)
2 {
3     if (filiere == NULL || filiere->premier_du_liste == NULL)
4     {
5         printf("Il n'y a aucune etudiant");
6         return;
7     }
8     Etudiant *i = filiere->premier_du_liste;
9     Etudiant *j;
10    Etudiant *major_minor = NULL; // Pointeur pour majoriser
        le i
11    // il nous faut des variable et une pointeur pour les
        echange fineaux
12    char tmp_nom[50];
13    char tmp_prenom[50];
14    int tmp_id;
15    int tmp_etat_inscription;
16    float tmp_moyenne_general;
17    Note *tmp_modules;
18    while (i != NULL && i->etudiant_suivant != NULL)
19    {
20        major_minor = i; // on suppose que le plus grand est
        le i
21        j = i->etudiant_suivant;
22        while (j != NULL)
23        {
24            if (j->moyenne_general > major_minor->
                moyenne_general)
25            {
26                major_minor = j; // j devient temporairement
                le plus grand
27            }
28            j = j->etudiant_suivant; // j passe a l etudiant
                suivant

```

```

29     }
30     if (major_minor != i)
31     {
32         strcpy(tmp_nom, i->nom);
33         strcpy(tmp_prenom, i->prenom);
34         tmp_id = i->numero_dossier;
35         tmp_etat_inscription = i->eta_inscription;
36         tmp_moyenne_general = i->moyenne_general;
37         tmp_modules = i->modules;
38
39         strcpy(i->nom, major_minor->nom);
40         strcpy(i->prenom, major_minor->prenom);
41         i->numero_dossier = major_minor->numero_dossier;
42         i->eta_inscription = major_minor->eta_inscription
43         ;
44         i->moyenne_general = major_minor->moyenne_general
45         ;
46         i->modules = major_minor->modules;
47
48         strcpy(major_minor->nom, tmp_nom);
49         strcpy(major_minor->prenom, tmp_prenom);
50         major_minor->numero_dossier = tmp_id;
51         major_minor->eta_inscription =
52             tmp_etat_inscription;
53         major_minor->moyenne_general =
54             tmp_moyenne_general;
55         major_minor->modules = tmp_modules;
56     }
57     i = i->etudiant_suivant;
58 }
59 }

```

3.0.9 Explication de la Fonctions du majorant_minorant

Cette fonction Permet d'afficher par ordre de mérite tout les étudiants de la filière du majorant à l'étudiant ayant la plus faible moyenne grâce à la trie par sélection :

```

1     while (i != NULL && i->etudiant_suivant != NULL)
2     {
3         major_minor = i; // on suppose que le plus grand est
4             le i
5         j = i->etudiant_suivant;
6         while (j != NULL)
7         {
8             if (j->moyenne_general > major_minor->
9                 moyenne_general)
10            {

```



```

9             major_minor = j; // j devient temporairement
               le plus grand
10         }
11         j = j->etudiant_suivant; // j passe a l etudiant
               suivant
12     }
13     if (major_minor != i)
14     {
15         strcpy(tmp_nom, i->nom);
16         strcpy(tmp_prenom, i->prenom);
17         tmp_id = i->numero_dossier;
18         tmp_etat_inscription = i->eta_inscription;
19         tmp_moyenne_general = i->moyenne_general;
20         tmp_modules = i->modules;
21
22         strcpy(i->nom, major_minor->nom);
23         strcpy(i->prenom, major_minor->prenom);
24         i->numero_dossier = major_minor->numero_dossier;
25         i->eta_inscription = major_minor->eta_inscription
               ;
26         i->moyenne_general = major_minor->moyenne_general
               ;
27         i->modules = major_minor->modules;
28
29         strcpy(major_minor->nom, tmp_nom);
30         strcpy(major_minor->prenom, tmp_prenom);
31         major_minor->numero_dossier = tmp_id;
32         major_minor->eta_inscription =
               tmp_etat_inscription;
33         major_minor->moyenne_general =
               tmp_moyenne_general;
34         major_minor->modules = tmp_modules;
35     }
36     i = i->etudiant_suivant;
37 }

1 void orde_alphabet(Filiere *filiere)
2 {
3     if (filiere == NULL || filiere->premier_du_liste == NULL)
4     {
5         printf("Il n'y a aucune etudiant\n");
6         return;
7     }
8
9     Etudiant *i = filiere->premier_du_liste;
10    Etudiant *j;
11
12    char tmp_nom[50];
13    char tmp_prenom[50];

```

```

14     int tmp_id;
15     int tmp_etat_inscription;
16     float tmp_moyenne_general;
17     Note *tmp_modules;
18
19     while (i != NULL && i->etudiant_suivant != NULL)
20     {
21         j = i->etudiant_suivant;
22
23         while (j != NULL)
24         {
25             if (strcmp(i->nom, j->nom) > 0)
26             {
27                 strcpy(tmp_nom, i->nom);
28                 strcpy(tmp_prenom, i->prenom);
29                 tmp_id = i->numero_dossier;
30                 tmp_etat_inscription = i->eta_inscription;
31                 tmp_moyenne_general = i->moyenne_general;
32                 tmp_modules = i->modules;
33
34                 strcpy(i->nom, j->nom);
35                 strcpy(i->prenom, j->prenom);
36                 i->numero_dossier = j->numero_dossier;
37                 i->eta_inscription = j->eta_inscription;
38                 i->moyenne_general = j->moyenne_general;
39                 i->modules = j->modules;
40
41                 strcpy(j->nom, tmp_nom);
42                 strcpy(j->prenom, tmp_prenom);
43                 j->numero_dossier = tmp_id;
44                 j->eta_inscription = tmp_etat_inscription;
45                 j->moyenne_general = tmp_moyenne_general;
46                 j->modules = tmp_modules;
47             }
48             j = j->etudiant_suivant; // j avance
49         }
50
51         i = i->etudiant_suivant; // i avance
52     }
53 }

```

3.0.10 Explication de la Fonctions orde_alphabet

Consernant cette fonction elle utilise une trie a bulle, et elle va permettre l'affichage de la liste par ordre alphabetique :

```

1     while (i != NULL && i->etudiant_suivant != NULL)
2     {

```

```

3         j = i->etudiant_suivant;
4
5         while (j != NULL)
6         {
7             if (strcmp(i->nom, j->nom) > 0)
8             {
9                 strcpy(tmp_nom, i->nom);
10                strcpy(tmp_prenom, i->prenom);
11                tmp_id = i->numero_dossier;
12                tmp_etat_inscription = i->eta_inscription;
13                tmp_moyenne_general = i->moyenne_general;
14                tmp_modules = i->modules;
15
16                strcpy(i->nom, j->nom);
17                strcpy(i->prenom, j->prenom);
18                i->numero_dossier = j->numero_dossier;
19                i->eta_inscription = j->eta_inscription;
20                i->moyenne_general = j->moyenne_general;
21                i->modules = j->modules;
22
23                strcpy(j->nom, tmp_nom);
24                strcpy(j->prenom, tmp_prenom);
25                j->numero_dossier = tmp_id;
26                j->eta_inscription = tmp_etat_inscription;
27                j->moyenne_general = tmp_moyenne_general;
28                j->modules = tmp_modules;
29            }
30            j = j->etudiant_suivant; // j avance
31        }
32
33        i = i->etudiant_suivant; // i avance
34    }

```

4 Modélisation du corps de notre projet : (int main)

C'est ici que nous faisons les appels des différentes fonctions pour donner vie à notre programme.

```

1 int main()
2 {
3     int votre_choix1 = 1;
4     int votre_choix2 = 1;
5     Etudiant *nouveau_etudiant = NULL;
6     Filiere ma_filiere;
7     ma_filiere.premier_du_liste = NULL;
8     ma_filiere.dernier_du_liste = NULL;

```

```

9      ma_filiere.nb_etudiants = 0;
10
11      printf("=====BIENVENUE!=====\n");
12      printf("Veillez donner le nom de votre filiere:\n");
13      scanf("%s", ma_filiere.nom_filiere);
14      while (votre_choix1 != 0)
15      {
16          nouveau_etudiant = inscrire_etudiants(&ma_filiere);
17          printf("Voulez vous ajouter ces note si oui tape (1) si non (0)\n");
18          scanf("%d", &votre_choix2);
19          if (votre_choix2 == 1)
20          {
21              note_etudiants(nouveau_etudiant);
22          }
23          printf("Voulez vous inscrire un nouveaux etudiant si oui tape (1) si non (0)\n");
24          scanf("%d", &votre_choix1);
25      }
26      liste_de_la_classe(&ma_filiere);
27      bulletin(&ma_filiere);
28      recherche_et_modification_etudiant(&ma_filiere);
29      supprimer_etudiant(&ma_filiere);
30      maximum(&ma_filiere);
31      minimum(&ma_filiere);
32      du_majorant_minorant(&ma_filiere);
33      printf("-----Classe trie par ordre de merite du meilleure au moins bon-----\n");
34      liste_de_la_classe(&ma_filiere);
35      orde_alphabet(&ma_filiere);
36      printf("-----Classe trie par ordre alphabetique-----\n");
37      liste_de_la_classe(&ma_filiere);
38
39      return 0;
40 }

```